Probabilistic Machine Learning (CS772A), Spring 2026
Indian Institute of Technology Kanpur
Homework Assignment Number 2

Student Name: Venkatesh Akula
Roll Number: 220109
Date: June 25, 2026

QUESTION
1

Q1.1 – Reconstruction Loss

Part 1: Gaussian log-likelihood ⇔ MSE

We have the decoder models $p_\theta(x|z) = \mathcal{N}(f_\theta(z),\, \sigma^2 I)$ where $x \in \mathbb{R}^D$. The log-likelihood is:

$$\begin{aligned}\log p_\theta(x|z) &= \log\left[\frac{1}{(2\pi\sigma^2)^{D/2}} \exp\left(-\frac{\|x - f_\theta(z)\|^2}{2\sigma^2}\right)\right] \\ &= -\frac{D}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\|x - f_\theta(z)\|^2 \end{aligned} \tag{1}$$

Maximizing $\log p_\theta(x|z)$ with respect to θ is equivalent to minimizing

$$\frac{1}{2\sigma^2}\|x - f_\theta(z)\|^2$$

since $-\frac{D}{2}\log(2\pi\sigma^2)$ is a constant w.r.t. θ. This is proportional to the **Mean Squared Error** (MSE) between x and $f_\theta(z)$:

$$\text{MSE} = \frac{1}{D}\|x - f_\theta(z)\|^2 = \frac{1}{D}\sum_{i=1}^{D}(x_i - f_\theta(z)_i)^2$$

Since σ^2 is constant, hence, maximizing the Gaussian log-likelihood is equivalent to minimizing MSE. □

Part 2: Deterministic decoder implications

When the decoder outputs only the mean $f_\theta(z)$ and ignores σ^2, we implicitly assume a *fixed, uniform* noise level across all pixels and all inputs. This means:

- Generated images are the *mean* of the conditional distribution - they tend to be **blurry**, especially when multiple plausible outputs exist for a given z.
- The model cannot express *per-pixel* or *input-dependent* uncertainty (heteroscedastic noise). Since the model does not cause uncertainty in the reconstruction, it may lead to overfitting on the training data.
- Reconstruction quality is averaged rather than sharp, since MSE penalizes all deviations equally and encourages hedging toward the mean.

Part 3: Bernoulli likelihood and Binary Cross-Entropy

Under an independent Bernoulli model, $p_\theta(x|z) = \prod_{i=1}^{D} \hat{x}_i^{x_i}(1-\hat{x}_i)^{1-x_i}$, the negative log-likelihood is:

$$-\log p_\theta(x|z) = -\sum_{i=1}^{D} [x_i \log \hat{x}_i + (1-x_i)\log(1-\hat{x}_i)]$$

This is exactly the **Binary Cross-Entropy (BCE)** loss.

Why is BCE more principled for MNIST:

1. MNIST pixels are nearly binary (black/white), essentially representing ink vs. no ink. Modeling each pixel as an independent Bernoulli is a better distributional assumption that respects this discrete nature, compared to a Gaussian with fixed variance.
2. BCE naturally constrains outputs to $[0, 1]$ (via sigmoid), matching the pixel range and providing proper probabilistic gradients for the data.
3. MSE treats pixels as continuous Gaussian variables, placing nonzero probability on values outside $[0, 1]$, which is physically inappropriate for discrete pixel intensities.

Q1.2 – Closed-Form KL Derivence

Given: $q_\phi(z|x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ and $p(z) = \mathcal{N}(0, I)$, both d-dimensional.

Starting from the definition:

$$D_{\text{KL}}(q\|p) = \int q(z) \log \frac{q(z)}{p(z)}\, dz = \mathbb{E}_q[\log q(z)] - \mathbb{E}_q[\log p(z)] \tag{2}$$

Step 1: Compute $\mathbb{E}_q[\log q(z)]$.

Since $q(z) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$:

$$\log q(z) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\sum_{j=1}^{d} \log \sigma_j^2 - \frac{1}{2}\sum_{j=1}^{d} \frac{(z_j - \mu_j)^2}{\sigma_j^2}$$

Taking expectation under q (noting $\mathbb{E}_q[(z_j - \mu_j)^2] = \sigma_j^2$):

$$\begin{aligned}\mathbb{E}_q[\log q(z)] &= -\frac{d}{2}\log(2\pi) - \frac{1}{2}\sum_{j=1}^{d} \log \sigma_j^2 - \frac{1}{2}\sum_{j=1}^{d} 1 \\ &= -\frac{d}{2}\log(2\pi) - \frac{1}{2}\sum_{j=1}^{d} \log \sigma_j^2 - \frac{d}{2}\end{aligned} \tag{3}$$

Step 2: Compute $\mathbb{E}_q[\log p(z)]$.

Since $p(z) = \mathcal{N}(0, I)$:

$$\log p(z) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\sum_{j=1}^{d} z_j^2$$

Taking expectation (noting $\mathbb{E}_q[z_j^2] = \mu_j^2 + \sigma_j^2$):

$$\mathbb{E}_q[\log p(z)] = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\sum_{j=1}^{d}(\mu_j^2 + \sigma_j^2) \tag{4}$$

Step 3: Subtract.

$$\begin{aligned} D_{\mathrm{KL}}(q\|p) &= (3) - (4) \\ &= \left[-\frac{1}{2}\sum_{j}\log\sigma_j^2 - \frac{d}{2}\right] - \left[-\frac{1}{2}\sum_{j}(\mu_j^2 + \sigma_j^2)\right] \\ &= -\frac{1}{2}\sum_{j=1}^{d}\log\sigma_j^2 - \frac{d}{2} + \frac{1}{2}\sum_{j=1}^{d}(\mu_j^2 + \sigma_j^2) \\ &= \frac{1}{2}\sum_{j=1}^{d}\left(\mu_j^2 + \sigma_j^2 - \log\sigma_j^2 - 1\right) \end{aligned} \tag{5}$$

Equivalently:

$$\boxed{D_{\mathrm{KL}}(q_\phi(z|x)\|p(z)) = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right)}$$

□

Probabilistic Machine Learning (CS772A), Spring 2026
Indian Institute of Technology Kanpur
Homework Assignment Number 2

Student Name: Venkatesh Akula
Roll Number: 220109
Date: June 25, 2026

QUESTION
2

Q2.1 – Reparameterization Trick

Implementation: Instead of sampling $z \sim \mathcal{N}(\mu, \text{diag}(\sigma^2))$ directly, we write:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

where $\sigma = \exp(0.5 \cdot \log \sigma^2)$ and $\odot$ denotes element-wise multiplication.

Why is it necessary:

If we use `z = Normal(mu, sigma).sample()`, the sampling operation is *non-differentiable* - PyTorch cannot backpropagate gradients through a stochastic sampling step. The reparameterization trick moves the stochasticity to ϵ (which doesn't depend on any learnable parameters), making z a deterministic, differentiable function of μ and σ. This allows gradients of the ELBO to flow through z back into the encoder parameters ϕ.

Q2.2 – Loss Function

Please check the submitted ipynb file. All the implementation details present there.

Q2.3 – Training loop

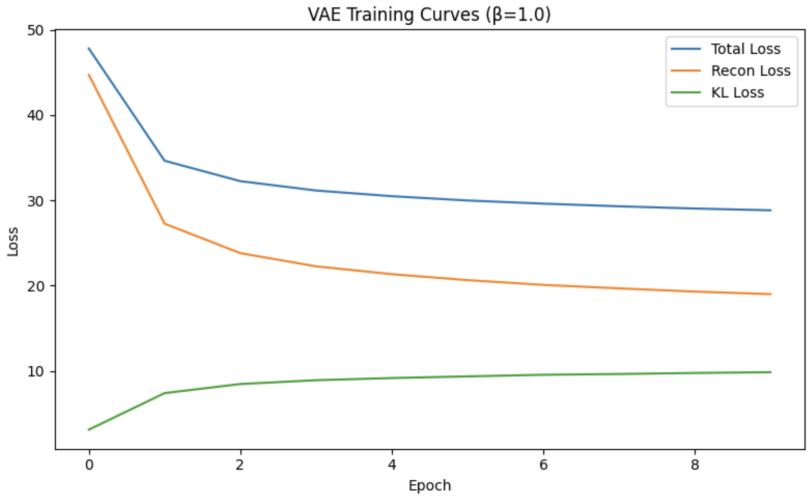


Figure 1: Loss plots

Q2.4 – Monte Carlo KL Analysis

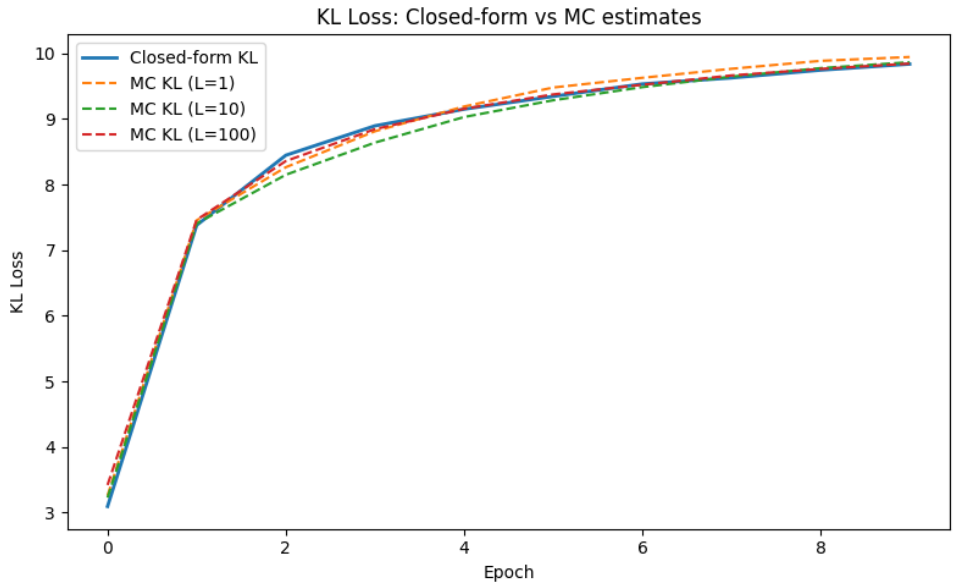


Figure 2: KL Loss

Observations on the Loss Curves:

- $L = 1$**:** The KL loss curve exhibits a highly noisy and jagged trajectory. Because each estimate relies on a single sample, the resulting gradient has extremely high variance.
- $L = 100$**:** The curve is drastically smoother and closely mirrors the closed-form baseline. While all curves ultimately converge to similar final values, a larger L provides a much more stable optimization path.

Why high variance is harmful during early training: During the initial phases of training, the encoder's distribution $q_\phi(z|x)$ is far from the prior, and the loss landscape is complex. Introducing high-variance gradient estimates from the KL term creates several compounding issues:

1. **Unstable Updates :** Noisy gradients cause erratic parameter updates, forcing the optimizer to "zigzag" inefficiently across batches rather than making consistent, direct progress.
2. **Risk of Local Minima:** High variance can cause the optimizer to take overly large steps in the wrong direction, potentially ejecting the model from promising regions or trapping it in poor local minima early on.
3. **Posterior Collapse:** The KL term dictates the regularization of the latent space. Wildly fluctuating KL gradients can trick the model into over-regularizing too quickly, causing the posterior to collapse before meaningful latent structures are learned.
4. **Hyperparameter Sensitivity:** Noisy gradients make tuning the learning rate incredibly difficult; a standard learning rate might cause divergence, while a safe, small learning rate will result in agonizingly slow convergence.

Why prefer the closed-form solution (even for large L):

1. **Zero Variance:** The analytical solution is exact. Even with $L = 100$, Monte Carlo estimation carries residual variance that injects unnecessary noise into your gradient updates.
2. **Compute Overhead:** MC requires drawing L samples and computing $2L$ log-probabilities per batch, whereas the closed-form evaluates instantly using basic arithmetic.
3. **Memory Scaling:** Because MC scales linearly, it increases the overall memory footprint and computation time by a factor of $\sim L$, wasting resources when an exact solution exists.

Student Name: Venkatesh Akula
Roll Number: 220109
Date: June 25, 2026

QUESTION
3

Q3.1 – Plots of Reconstruction and KL Loss curves

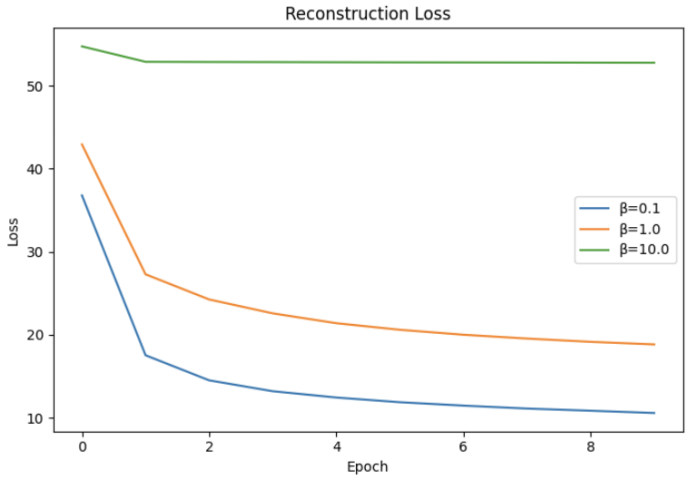


Figure 3: Reconstruction Loss

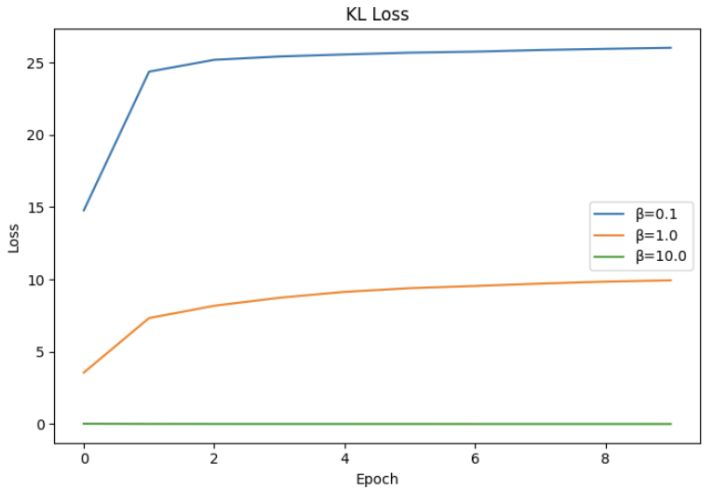
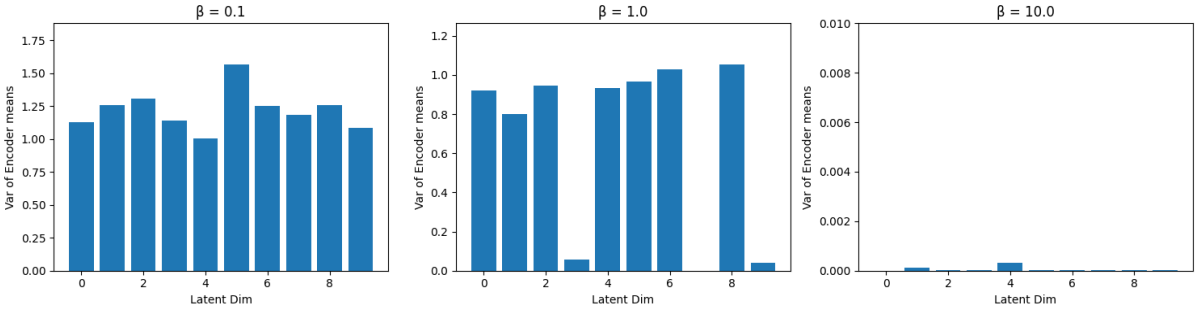


Figure 4: KL Loss

Observation: The plots clearly illustrate the β-VAE trade-off: a lower β (0.1) yields better reconstruction at the cost of high KL divergence, while a large β (10.0) over-regularizes the

model, instantly driving the KL loss to zero (posterior collapse) while leaving the reconstruction loss stagnant and high.

Q3.2 – Posterior Collapse Analysis



Observations Across β Values: Comparing the active dimensions across the three models reveals the trade-offs of the β parameter:

- **$\beta = 0.1$ (Weak Regularization):** The model places very little penalty on the latent space structure, resulting in high variance across most dimensions. The encoder over-utilizes the latent space, acting almost like a standard autoencoder without enforcing a good prior.
- **$\beta = 1.0$ (Balanced Trade-off):** A standard VAE where only a meaningful subset of dimensions remain active. The model naturally prunes unnecessary dimensions (variance ≈ 0) while keeping enough active dimensions to encode distinct input features.
- **$\beta = 10.0$ (Strong Regularization):** Almost every single latent dimension collapses to a near-zero variance across the test set. The encoder maps nearly all inputs to the exact same generic region. This is the classic signature of **posterior collapse**.

Why a large β incentivizes the encoder to ignore the input: The training objective is $\mathcal{L}_\beta = \text{Recon} + \beta \cdot \text{KL}$. When $\beta = 10.0$, the penalty for deviating from the prior $\mathcal{N}(0, I)$ becomes overwhelmingly dominant compared to the reconstruction loss. The path of least resistance for the optimizer to minimize this heavily weighted loss is to aggressively drive the KL divergence to zero. To do this, the encoder learns to output $\mu \approx 0$ and $\sigma^2 \approx 1$ regardless of the specific input x, completely ignoring the data it is being fed.

Why the decoder receives no useful signal: When the encoder forces $q_\phi(z|x) \approx p(z)$ for every input, the latent vector z passed to the decoder is practically just random noise drawn from the standard normal distribution.

- **No Information Transmission:** Because z is identical (statistically) for every x, it carries zero specific information about the original input. The bottleneck becomes a brick wall, severing the connection between the encoder and decoder.
- **Degenerate Solution & Poor Reconstruction:** Since the decoder cannot distinguish between different inputs based on z, it resorts to a degenerate strategy. To minimize the reconstruction error across the entire dataset without any input-specific clues, it simply learns to output the global "average" of the training data (resulting in a blurry, unrecognizable mean image).

Probabilistic Machine Learning (CS772A), Spring 2026
Indian Institute of Technology Kanpur
Homework Assignment Number 2

Student Name: Venkatesh Akula
Roll Number: 220109
Date: June 25, 2026

QUESTION

4

Q4.1 – Interpolation Analysis

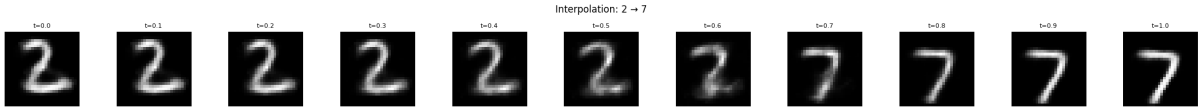


Figure 5: Interpolation image

Key Observations:

- **Smooth Transition:** The interpolated images show a smooth, gradual transition from 2 to 7 with no abrupt jumps or random noise.
- **Semantic Hybrids:** In the middle steps (around $t \approx 0.3 - 0.7$), the images look like plausible hybrid digits. For example, the bottom stroke of the 2 gradually straightens out while the top curve smoothly flattens into the horizontal bar of the 7.
- **Continuity**: We observe that at no point the interpolation produce random noise or unrecognizable patterns.

Role of KL Regularization vs. Standard AE:

- **Preventing Holes:** Without the KL penalty, a standard Autoencoder can map different classes to arbitrary, widely disconnected regions of the latent space. The empty spaces between these clusters are "dead zones" the decoder never sees during training. Consequently, interpolating through them produces meaningless, garbage images.
- **Dense Packing & Overlap:** The KL term fixes this by forcing the encoder's distributions $q_\phi(z|x)$ to stay close to the prior $\mathcal{N}(0, I)$. This packs all the encoded data tightly around the origin, forcing the distributions of different digits to naturally overlap.
- **Continuous Manifold:** Because the latent space is densely occupied and overlapping, any linear path between two points stays inside regions the decoder has actually been trained to interpret. This guarantees that intermediate latent codes z_t decode into structurally valid images.

Q4.2 – 2D Visualization (Bonus)

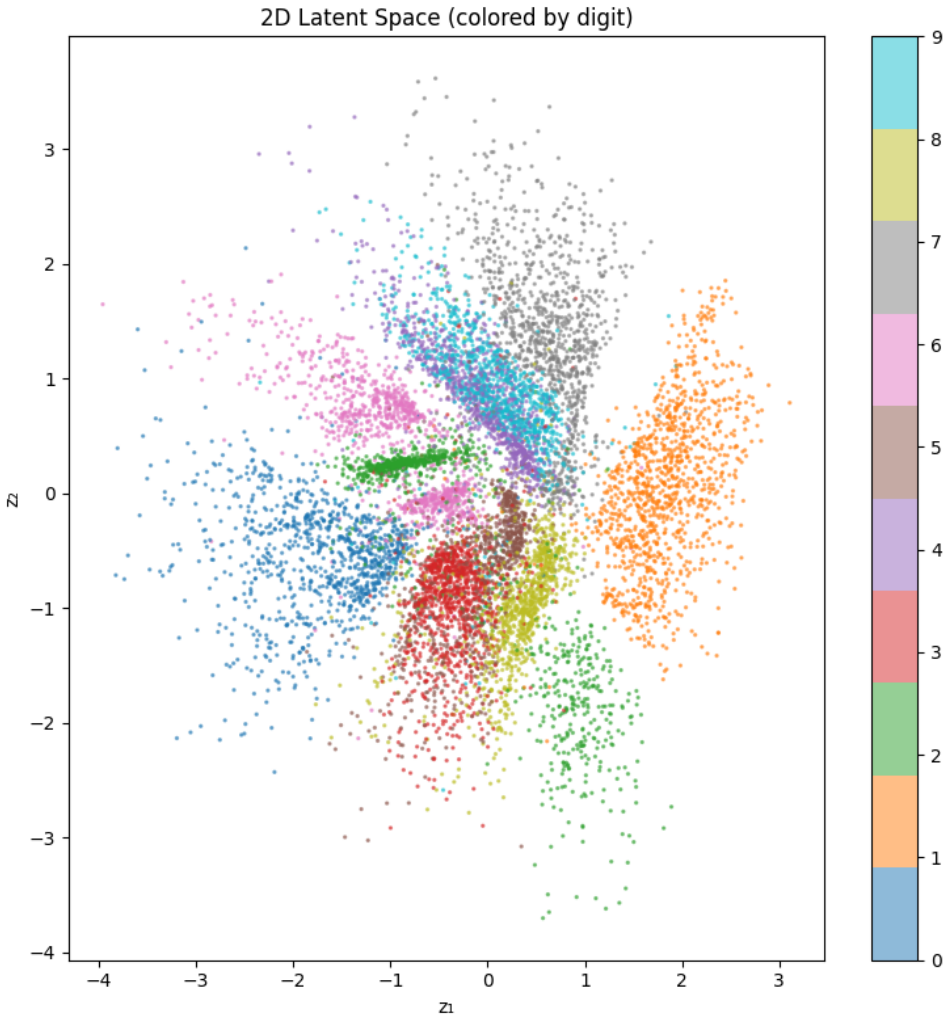


Figure 6: Latent space visualisation

My Key Observations:

The plots reveal 3 key patterns as follows:

- **Separable but Centered:** Digit classes form identifiable, roughly separable clusters, but the macro-distribution forms a large cluster centered at the origin. This confirms the KL term's $\mathcal{N}(0, I)$ regularization is working.
- **Semantic Clustering:** Visually similar digits (e.g., 3 and 8, or 4 and 9) tend to share adjacent regions and have heavily overlapping boundaries.
- **Expected Overlap:** Perfect separation is impossible because 10 complex classes are squeezed into a severe 2-dimensional bottleneck. Furthermore, the KL term actively prevents the model from pushing the clusters infinitely far apart to artificially separate them, resulting in the expected compromises and overlaps.